



Figure 2: Model representation

The simple way to implement a model is through mathematical symbols. Let us consider ‘I’ to be an instance of data such as a feature vector. The related label associated with ‘I’ is ‘f(I)’. We can use the term ‘y=f(I)’ because it is easy to write. In context with any other example of classification, ‘I’ is a hundred-dimensional vector of different related vectors and ‘y’ is a pair of values that represent an outcome.

The model prediction of ‘I’ is denoted as ‘g(I)’. The model will have parameters that can be used and modelled either by humans or automatically. The parameter is represented by a vector ‘θ’. The combination of all these, ‘g(I|θ)’, represents a model.

Unsupervised learning models the data without labels or responses. Machine learning uses two powerful tools that learn from data alone — clustering and dimensionality reduction. Clustering is the process of collecting and dividing data into groups of similar items. Dimensionality reduction is used to manipulate data in order to view it in an easier way.

Reinforcement learning interacts with the environment to learn which pair or collection of action leads to an efficient outcome.

Machine learning uses powerful tools that are standardised, robust, and have high performance and scalability. This is where TensorFlow comes into the picture.

TensorFlow essentials

The machine learning framework created by Google is TensorFlow. This is used to “design, build and train

machine learning models.” TensorFlow uses various libraries to do numerical computations, which are executed with a data flow graph. The graph contains nodes and edges, where nodes represent “mathematical operations and edges represent the data, which are generally multi-dimensional arrays or tensors, which are communicated between the edges.” TensorFlow enables developers to develop and deploy the models with ease and use them in machine learning. Automatic differentiation is one of the important properties of TensorFlow, since it makes it easier to implement the neural network concept of back-propagation.

Feature vectors are used predominantly in machine learning. A matrix represents a list of vectors, in which every bit of data contains a feature vector. Every column of a matrix represents a feature vector.

The syntax to represent a matrix is a vector of vectors in TensorFlow with the same length. The syntax for a tensor is nested vectors. The notion of scalar is rank 0 tensors, rank 1 tensors (i.e., vectors) and rank 2 tensors (i.e., matrices).

TensorFlow’s main data structure — tensors

Data management in TensorFlow is based on ‘tensors’. The fundamental concept of tensors “originates from the field of mathematics which is developed as a generalisation under the category of linear algebra in terms of vectors and matrices.” A tensor is described as a typed, multi-dimensional array with added operators modelled into the tensor object.

TensorFlow uses tensor data structure for representing all the data,

since only the objects of the tensor type will be passed between the nodes in the computation graph.

Properties of tensors

Tensor rank: The dimensional aspect of a tensor is represented by tensor rank, which is different from a matrix rank. It represents the “quantity of dimensions” in which the tensors live. A rank 1 tensor is equivalent to a vector and comprises a matrix too. For a rank 2 tensor, any component can be approached with the syntax `t[i, j]`. For a rank 3 tensor, you will need to address a component with `t[i, j, k]`, and so on.

Tensor shape: There is always a distinct connection between tensors and shapes. Rank 1, rank 2, rank 3 tensors have a shape of dimension 1, dimension 2 and dimension 3, respectively. An alternate way to represent a rank 3 tensor is a list that contains “matrices with the same shape.”

Tensor types: Tensors are of different types such as `tf.float32`, `tf.float64`, `tf.int32`, and `tf.int64`.

Handling the computation workflow

TensorFlow always behaves imperatively. It is recommended to install it under Anaconda Python.

Initialize interactive TensorFlow session

```
>>> import tensorflow as tf
>>> tf.InteractiveSession()
<tensorflow.python.client.session.InteractiveSession>
```

Create a zeros tensor

```
>>> tf.zeros(2)
<tf.Tensor 'zeros: 0' shape=(2,) dtype=float32>
```

Evaluate the value of tensor

```
>>> a = tf.zeros(2)
>>> a.eval()
array([ 0.,  0.], dtype=float32)
```

Creating constant tensors

```
>>> a = tf.constant(2)
>>> a.eval()
2
```